DLDCA 11 Inlab: Does your head hurt?

Week 11

November 4, 2026

"Don't try to understand it. Feel it."

∼ Nolan, on computer architecture lab

This lab is ungraded

This is the last lab :(. We hope you enjoyed learning about digital logic (Verilog <3) and computer architecture, whichever parts you liked the most (performing assembly vs simulating performance???). This lab is **ungraded**, so treat it as practice. If you do Task 2 right now, it will help you with Outlab 10. Task 3 is what we planned to cover in Lab 10 itself, so it is here for you. Task 1 is performance revision.

Introduction

Three tasks, one from each part of the back half of the course:

- **Task 1 (performance).** Take a correct but slow 2D convolution and make it fast, one technique at a time: a better loop order, SIMD, then unrolling. Measure each step with `perf`.
- **Task 2 (simulator).** Extend the SRRIP replacement policy into DRRIP, which picks its insertion policy by watching which one misses less. A head start on the outlab, where you write SRRIP from scratch.
- **Task 3 (simulator).** Make the LLC exclusive of the L2C. This is the Week 10 inlab task, included again as revision.

Task 1 is native C++ measured with `perf`. Tasks 2 and 3 run on ChampSim, the same simulator and traces as Weeks 9 and 10. The two halves set up separately; each has its own folder and its own `make`.

Task 1 setup: `g++` and `perf`

Where to run Task 1

`perf` needs the hardware counters of a real CPU. Use a **Software Lab machine over ssh** or a **native Linux machine**. On a virtual machine or WSL, `perf stat` prints `<not supported>` for cycles and the task cannot be measured.

Everything for Task 1 is in `task1/`. You need `g++` and `perf`:

```
$ cd task1
$ g++ --version
$ perf --version
```

Tasks 2 and 3 setup: ChampSim

These two tasks run on ChampSim, exactly as in Weeks 9 and 10. If your setup from those weeks still works, you are ready.

Step 1: Tools. You need `g++` (C++17), `make`, `python3` and `xz`. Each should print a version:

```
$ g++ --version
$ make --version
$ python3 --version
$ xz --version
```

Step 2: Traces. The same four traces as Weeks 9 and 10: `soplex`, `bwaves`, `mcf` and `gcc`. On the lab machines they are in `/opt/CS231_Traces`. On your own machine, point the makefile at your copy:

```
$ export CS231_TRACE_DIR=/path/to/your/trace/folder
```

If `CS231_TRACE_DIR` is not set, the makefile looks in `/opt/CS231_Traces`. A trace is found by name: `trace=soplex` picks the file matching `*soplex*.xz`.

Step 3: Check it builds. From the lab folder, build ChampSim with `lru` at the LLC and run it once (about a minute):

```
$ make lru
...
CPU 0 cumulative IPC: ...
LLC MPKI: ...
```

A modified ChampSim. The `Champsim/` folder is the one from Weeks 9 and 10 with the built-in SRRIP, DRRIP and SHiP policies removed, so you write DRRIP yourself. `make task2` and `make task3` build it for you from the files in each task folder; you do not edit `Champsim/` itself.

The replacement policy interface (Task 2)

A replacement policy plugs into ChampSim through three functions, the same ones you used in Week 9:

`initialize_replacement()` called once before the run; set up your per-block state here.

`find_victim(..., set, ...) -> way` called when a set is full; return the way to evict.

`update_replacement_state(..., set, way, ..., type, hit)` called after every hit (`hit = 1`) and every fill (`hit = 0`). `type` is `LOAD`, `RFO`, `PREFETCH`, `WRITEBACK` or `TRANSLATION`.

Task 2's policy runs at the **LLC**. Task 3 instead edits `cache.cc` directly; it names the functions it changes and the Week 10 PDF has the full map.

The numbers we check (Tasks 2 and 3)

- **IPC**, from `CPU 0 cumulative IPC`. Higher is better.
- **LLC MPKI** and **L2C MPKI**: demand misses (`LOAD` + `RFO`) per 1000 instructions, which the makefile computes for you.

ChampSim is deterministic, so a correct solution reproduces our numbers exactly, on any machine.

Building and testing

`make` commands (run from the lab folder)

`make task1`	build every Task 1 stage (then measure them in `task1/`)
`make task2`	build ChampSim with your `task2/drrip.cc` at the LLC; run on `soplex`
`make task3`	apply `task3/task3.patch`, use your `task3/cache.cc`; run on `soplex`
`... trace=mcf`	use another trace: `soplex`, `bwaves`, `mcf` or `gcc`
`... long=1`	longer run (20M + 25M) instead of the quick one
`make lru`	ChampSim with `lru` at the LLC: the baseline for Task 2
`make check`	run every task and compare with the reference
`make check-task2`	one task (also `check-task1`, `check-task3`)
`make clean`	delete everything that was built

Task 1 has its own commands inside `task1/` (`make conv_simd`, `make check`, and so on); see the task.

Structure of submission/templates

Edit only the files marked `(TODO)`.

```
your-roll-no/
|- task1/
    |- conv_reorder.cpp (TODO)
    |- conv_simd.cpp (TODO)
    |- conv_unroll.cpp (TODO)
    |- questions.txt (TODO)
|- task2/
    |- drrip.cc (TODO)
|- task3/
    |- cache.cc (TODO)
    |- task3.patch
    |- apply_patch.sh
    |- expected_output.txt
```

`./build_archive.sh` packs these into `your-roll-no.tar.gz`. Nothing is graded, so submission is only so the TAs can see your work if you want feedback.

Tasks

Task 1: Make a Convolution Fast

A 2D convolution slides a small `K×K` kernel over an image: each output pixel is the weighted sum of the `K×K` input pixels around it. It is the core of image filters and of every convolutional neural network, and it is almost all multiply-add. The arithmetic is fixed; how fast it runs depends on how well the loops match the machine.

You are given a correct, slow version in `conv_naive.cpp`. You write three faster versions, one technique at a time, and measure each with `perf`. Every version computes the same result; the harness checks it against the naive output.

The computation. Let I be the $H \times W$ input image, G the $K \times K$ kernel (K odd) and $p = (K-1)/2$. The input is zero-padded by p on every side into P, of size $(H+2p) \times (W+2p)$:

$$P[r][c] = \begin{cases} I[r-p][c-p] & \text{if } p \le r < H+p \text{ and } p \le c < W+p, \\ 0 & \text{otherwise.} \end{cases}$$

The output O has the same size as the input, $H \times W$:

$$O[y][x] = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} P[y+i][x+j] \cdot G[i][j], \qquad 0 \le y < H,\ 0 \le x < W.$$

The kernel is not flipped (strictly, this is a cross-correlation). All three arrays are row-major `float`: `in[r*(W+2p) + c]` $= P[r][c]$, `out[y*W + x]` $= O[y][x]$ and `ker[i*K + j]` $= G[i][j]$. The padding means no boundary checks are needed. W is a multiple of 4.

Size you are measured on: exactly $H = W = 2048$ and $K = 3$. Each binary runs the stage 20 times, times each run separately, and reports the **median** of the 20 times, so one slow run does not change the result. This is what every binary runs when you give it no arguments (`make conv_<stage>` or `./conv_<stage>.out`). Use this size for every `perf stat` run and every number in `questions.txt`. Other sizes are only for your own experiments.

The shared signature (do not change it) is:

```
void conv(const float* in, float* out, const float* ker, int H, int W, int K);
```

The full data layout is documented at the top of `convolution.h`.

The stages. Each is one file, built into its own binary so you can measure it on its own.

`conv_naive.cpp`	**given.** One output pixel at a time; one running sum swept over the whole kernel.
`conv_reorder.cpp`	**a better loop order.** Work on 4 neighbouring output columns together, with 4 independent running sums, writing each output once.
`conv_simd.cpp`	**SIMD.** Those 4 sums are 4 floats side by side: one SSE register. One instruction now does all 4 columns.
`conv_unroll.cpp`	**unrolling.** Keep 4 SSE accumulators (16 columns) with 4 independent add chains, so several adds are in flight at once.

Why each helps.

- **Loop order.** In the naive version each `acc +=` waits for the one before it: a single long dependency chain. Keeping a few sums for neighbouring columns breaks it into independent chains, and the kernel weight loaded once is reused across them.
- **SIMD.** One `_mm_add_ps` adds four floats at once, so the inner loop runs about a quarter of the instructions.
- **Unrolling.** A single accumulator still forms one chain. Several accumulators let the CPU overlap independent adds and hide their latency. Each accumulator uses a register, so keep the unroll small: too many spill to memory and the gain is lost. Four fit easily in the 16 SSE registers.

Build, run, measure. From `task1/`:

```
$ make conv_naive            # build and run one stage (2048x2048, K=3)
$ make conv_simd args="1024 1024 5"   # custom H W K [reps]
$ make check                 # build and run every stage, PASS/FAIL and time
```

Each binary prints its median time, throughput and a `PASS`/`FAIL` against the reference.

Run perf on every stage

A faster time tells you *that* a stage is faster. `perf stat` tells you *why*. Once your stages pass, build all four and run this exact command on each binary. It prints only the counters you need:

```
$ make all
$ EV=task-clock,cycles,instructions,branches,branch-misses
$ EV=$EV,L1-dcache-loads,L1-dcache-load-misses
$ for s in conv_naive conv_reorder conv_simd conv_unroll; do
>   perf stat -e $EV taskset -c 1 ./$s.out
> done
```

To measure one stage, run `perf stat -e $EV taskset -c 1 ./conv_simd.out`. If `perf` does not print `insn per cycle`, compute the IPC yourself as `instructions / cycles`.

How to read the numbers. The run time is the cycle count, and *cycles* = *instructions*/*IPC*. A stage can therefore get faster in two ways: it runs fewer instructions, or it runs more instructions per cycle. For each stage, find out which of the two happened, then find the counter that explains it. Ask what one trip through the innermost loop costs in loads, multiply-adds and branches, and how many output pixels it produces. Also look at what does *not* change between stages. A counter that stays the same in every stage cannot be the reason for a speedup.

Every binary also runs the harness: one reference convolution and the result check. The harness is the same in all four stages, so compare the stages with each other rather than reading the absolute counts.

Questions. Answer the questions in `task1/questions.txt`. For each stage, compared with `conv_naive`, say which `perf stat` counters improved and explain why that stage's code changes them.

Reference speedups

One machine (a laptop, one core, `g++ -O2 -fno-tree-vectorize`), `2048 x 2048`, `K=3`, median of 20 runs. **Your times and speedups will differ by machine and compiler.** This is a practice task: nothing checks your numbers against this table. What matters is that each stage is faster than the one before it, and that you can explain why from `perf stat`.

stage	time	speedup vs naive
`conv_naive`	11.1 ms	1.0×
`conv_reorder`	6.5 ms	1.7×
`conv_simd`	4.3 ms	2.6×
`conv_unroll`	2.3 ms	4.8×

Task 2: DRRIP at the LLC

This task is Outlab 10's SRRIP plus one idea, *set dueling*, that lets the cache choose its own insertion policy. You write the SRRIP part yourself, exactly as in Outlab 10 Task 1, so doing this task now is a head start on the outlab.

Recap: SRRIP. Each block carries a re-reference prediction value (RRPV), a 2-bit number from 0 to maxRRPV = 3. An RRPV of 0 means "used again soon" and 3 means "not for a long time". SRRIP is three rules on this number:

```
insertion (fill):  RRPV[new block] = maxRRPV - 1    # long, not the longest
promotion (hit):   RRPV[block] = 0                  # reused: keep it
victim selection:
    loop:
        if some block in the set has RRPV == maxRRPV:
            return the first such way
        for every block in the set: RRPV = RRPV + 1  # age the set
```

A new block enters at maxRRPV − 1, one step from eviction, and earns a better value only by getting a hit. The writeback rule is the same as in the outlab: a writeback hit is not a reuse (leave the RRPV alone), and a writeback fill has the lowest priority (maxRRPV).

The problem SRRIP has. Inserting at maxRRPV−1 is a good guess when blocks are reused soon. But some workloads *thrash*: they stream through far more data than the cache holds, and almost nothing is reused before it is evicted. There, inserting near the front just evicts a block that *was* about to be reused. For those workloads a better rule is BRRIP (Bimodal RRIP): insert almost everything at the distant end (maxRRPV), and only once every so often (say 1 in 32 fills) at maxRRPV − 1. That keeps a small, stable part of the working set instead of thrashing the whole set.

So SRRIP is better for some workloads and BRRIP for others, and you do not know which in advance. DRRIP (Dynamic RRIP, Jaleel et al., ISCA 2010) decides at run time.

Set dueling. Hold a contest between the two policies inside the same cache:

- Pick a few **leader sets** that always use SRRIP, and a few that always use BRRIP. Here, fixed by set index: 32 SRRIP leaders and 32 BRRIP leaders.
- Keep one saturating counter, `PSEL` (10 bits, 0 to 1023), that tracks which group of leaders is doing better.
- Every other set, the **followers**, copies whichever policy `PSEL` currently favours.

The leaders are a live, always-on experiment; the followers take the experiment's verdict.

Implementing it. Victim selection and promotion do not change from SRRIP. Only the insertion of a demand fill changes, and it needs just two pieces of logic.

(A) An `if` that picks the policy for this set.

- An SRRIP leader set uses SRRIP; a BRRIP leader set uses BRRIP.
- A follower uses BRRIP if `PSEL` is in its upper half, else SRRIP.

(B) A rule that trains `PSEL` on the leaders. A demand miss in a leader set is a point against that set's policy, so move `PSEL` one step away from it (and keep it inside 0 to 1023):

- miss in an SRRIP leader ⇒ `PSEL++` (lean towards BRRIP);
- miss in a BRRIP leader ⇒ `PSEL−−` (lean towards SRRIP).

Followers do not train `PSEL`. Over time `PSEL` drifts towards whichever policy is missing less, and the followers follow.

For the BRRIP insertion itself, use a counter so that 1 fill in 32 enters at maxRRPV − 1 and the rest at maxRRPV.

What to do. Complete the `// TODO` markers in `task2/drrip.cc`. The file has the same layout as `srrip_llc.cc` from Outlab 10 Task 1.

- **TODO 1 to 4** are plain SRRIP: the RRPV structure, its initial values, victim selection, and promotion with the writeback rule.
- **TODO 5** picks the insertion policy of a demand fill and trains `PSEL` (pieces (A) and (B) above).
- **TODO 6** inserts the demand fill: SRRIP or BRRIP, with the 1-in-32 counter.

The DRRIP parameters (`PSEL`, `BRRIP_INTERVAL` and the leader-set functions) are given at the top of the file; do not change them, or your numbers will not match. Then:

```
$ make task2                 # build and run on soplex
$ make check-task2           # compare all four traces with the reference
$ make lru trace=soplex      # the LRU baseline, to compare against
```

What to expect. DRRIP picks the better of SRRIP and BRRIP per workload. That is not automatically better than LRU: on `mcf` the whole SRRIP family trails LRU, and DRRIP matches SRRIP there rather than beating LRU. What set dueling buys you is avoiding the *worst* case of either SRRIP or BRRIP alone, workload by workload. On `soplex` it raises IPC over LRU.

Reference numbers (`make check-task2`, 1M + 5M)

Your DRRIP must reproduce these exactly (ChampSim is deterministic). The LRU column is `make lru trace=[trace]` at the same lengths, for comparison.

trace	IPC (LRU)	IPC (DRRIP)	LLC MPKI (DRRIP)
`soplex`	0.3344	0.368691	30.0394
`bwaves`	0.2320	0.234333	38.7166
`mcf`	0.6470	0.609917	8.6528
`gcc`	0.1140	0.114033	68.9560

Task 3: An Exclusive LLC

This task is copied unchanged from the Week 10 inlab (Task 3). It involves modifying ChampSim's `cache.cc`, not writing a replacement policy. If you finished it in Week 10, you can reuse your `cache.cc`.

In the given ChampSim the hierarchy is NINE: every level keeps a copy of every line it fetches, so after a quick run of `soplex`, 8191 of the 8192 lines in the L2C are also in the LLC. The LLC spends 512 KB of its 2 MB on lines the L2C already has.

In this task you make the LLC **exclusive** of the L2C: a line is in the L2C or in the LLC, never in both. Together they then hold 2.5 MB of different lines. The LLC becomes a victim cache for the L2C. A line comes from DRAM straight into the L2C, goes down to the LLC when the L2C evicts it, and comes back up to the L2C on an LLC hit.

The patch. An exclusive LLC moves lines up and down, and a dirty line must stay dirty on the way. In the given ChampSim a packet has no dirty bit, so `task3/task3.patch` adds one (`PACKET::dirty`) and makes caches fill, write back and answer with it. With only the patch, ChampSim gives exactly the same numbers as before. The patch also makes `main.cc` print one line before the statistics, the number of valid L2C lines that also have a valid copy in the LLC:

```
L2C LINES ALSO IN LLC: 8191 of 8192
```

`make task3` applies the patch for you: it copies `Champsim/` to `.task3_champsim/`, applies the patch, puts your `task3/cache.cc` in place of `src/cache.cc`, builds and runs it. `task3/cache.cc` already contains the patch's changes to `cache.cc`. To patch another ChampSim folder by hand, run the given script on it, then edit its `src/cache.cc`:

```
$ ./task3/apply_patch.sh ~/Desktop/ChampSim
patched /home/[you]/Desktop/ChampSim
```

Your edits. You submit `task3/cache.cc`. It has an empty `// TODO:` at each place to edit, with a hint above it, and this flag at the top:

```
const bool EXCLUSIVE_LLC = true;
```

The code in `cache.cc` runs for every cache, and `NAME` tells you which one: `LLC` or `cpu0_L2C`. Guard every change with `EXCLUSIVE_LLC` (and `NAME` where it names a cache), so that with the flag set to `false` ChampSim behaves exactly as before.

Step 1: A bypass still returns its data. *Where:* `handle_fill` and `filllike_miss`.

Step 2 makes the LLC bypass its fills: the line goes up without taking a way (`way == NUM_WAY`). The given `handle_fill` sends data up only when the line took a way, so a bypass would never answer the L2C and the simulation would stop with `DEADLOCK!`. Send the data up for a bypass too, and skip the replacement update in `filllike_miss`, since a bypass has no way to update.

Step 2: A line from DRAM goes to the L2C only. *Where:* `handle_fill`, after the victim way is chosen.

An exclusive LLC holds only lines the L2C has evicted. When DRAM answers an LLC miss, the LLC bypasses the fill.

Step 3: The L2C sends every victim to the LLC. *Where:* `filllike_miss`, after `evicting_dirty` is set.

The LLC now gets its lines only from L2C evictions, so a clean victim must go down too, not only a dirty one. `evicting_dirty` decides whether the victim is sent; in the L2C, send every valid victim. The patch puts the victim's dirty bit in the packet, so a clean line arrives in the LLC clean.

Step 4: An LLC hit moves the line up. *Where:* `readlike_hit`, before the response is sent.

The L2C stores the line when the response arrives, so the LLC drops its copy. If the line is dirty, the L2C now owns the dirty data: put the dirty bit in the response (`handle_pkt`), then invalidate the LLC's block.

Check each step. Quick runs on `soplex` (`make task3`) give these numbers after each step. After step 4, `L2C LINES ALSO IN LLC` is 0. `make check-task3` prints this count for each trace too. In its shorter runs a few lines (3 on `soplex` and `mcf`) stay in both: the L2C asked for a line again while its victim was still in the LLC's write queue, so the LLC answered from the queue and then stored the victim as well. The reference solution gives the same count.

after	L2C lines also in LLC	IPC	L2C MPKI	LLC MPKI
given `cache.cc`	8191	0.317071	35.6121	28.6951
step 1	8191	0.317071	35.6121	28.6951
step 2	5800	0.383673	35.6123	30.1266
step 3	3899	0.325360	35.6121	54.0824
step 4	0	0.334986	35.6120	61.6739

Reference numbers: IPC, L2C MPKI and LLC MPKI

Long runs (`make task3 long=1 trace=[trace]`) on the lab machines give these numbers. **Your IPC, L2C MPKI and LLC MPKI must match the `exclusive LLC` rows.** The `given (NINE)` rows are the unchanged ChampSim, the same as `make lru`. The LLC MPKI doubles because every L2C victim is now an LLC write, yet the IPC goes up: `soplex` gains 6% and `mcf` 2%.

trace	cache.cc	IPC	L2C MPKI	LLC MPKI	L1D miss latency
`soplex`	given (NINE)	0.323267	34.0741	27.6852	310.9
	exclusive LLC	0.343337	34.0737	58.9574	284.2
`bwaves`	given (NINE)	0.243644	26.1799	26.1785	1400.8
	exclusive LLC	0.245052	26.1799	52.3575	1374.2
`mcf`	given (NINE)	0.409243	16.9194	10.1910	86.3
	exclusive LLC	0.418539	16.9193	26.3026	80.2
`gcc`	given (NINE)	0.113813	69.9909	69.9792	113.5
	exclusive LLC	0.113824	69.9909	139.9674	113.5

```
$ make task3 long=1
Building .task3_champsim/bin/champsim with task3.patch and task3/cache.cc
.task3_champsim/bin/champsim --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
L2C LINES ALSO IN LLC: 0 of 8192
CPU 0 cumulative IPC: 0.343337 instructions: 25000002 cycles: 72814766
cpu0_L2C TOTAL     ACCESS:    1434598  HIT:     582755  MISS:     851843
cpu0_L2C LOAD      ACCESS:     884472  HIT:     161938  MISS:     722534
cpu0_L2C RFO       ACCESS:     149420  HIT:      20417  MISS:     129003
cpu0_L2C WRITEBACK ACCESS:     400094  HIT:     399962  MISS:        132
LLC TOTAL     ACCESS:    1703547  HIT:     229612  MISS:    1473935
LLC LOAD      ACCESS:     722530  HIT:     175823  MISS:     546707
LLC RFO       ACCESS:     129001  HIT:      53565  MISS:      75436
LLC WRITEBACK ACCESS:     851842  HIT:        130  MISS:     851712
cpu0_L2C MPKI: 34.0737
LLC MPKI: 58.9574
```

A note on open-ended changes. A change like this one can be written in several ways, and different versions can all be correct. This one is small enough that everyone ends up with the same code, and `make check` expects our numbers exactly. In general, do not worry if your code is not identical to someone else's.